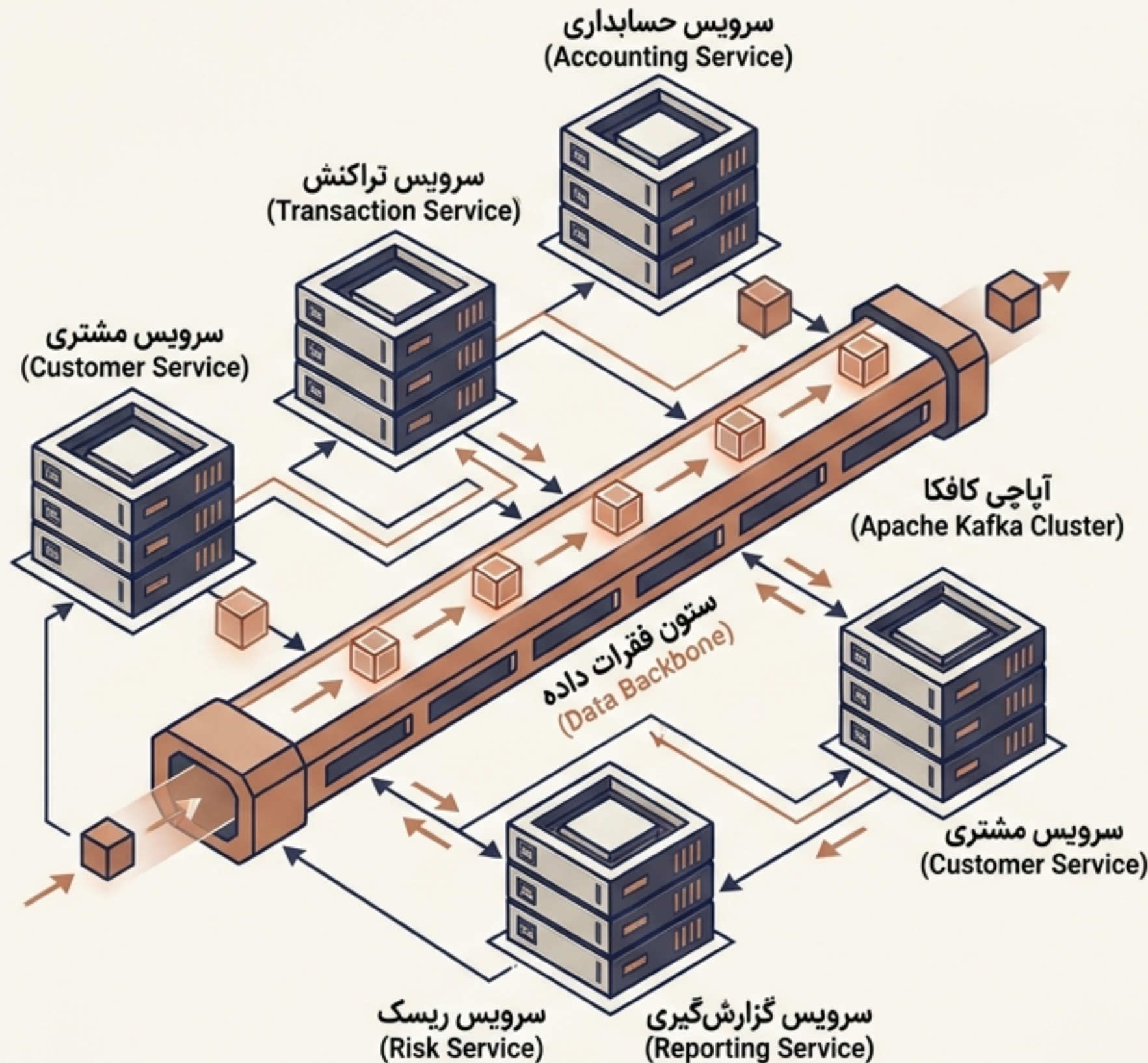


آپاچی کافکا در سیستم‌های مالی: از معماری پایه تا الگوهای پیشرفته



کافکا در سیستم‌های بانکی صرفاً یک صف پیام نیست؛ بلکه ستون فقرات غیرقابل‌تغییر و شریان اصلی میکروسرویس‌هاست.

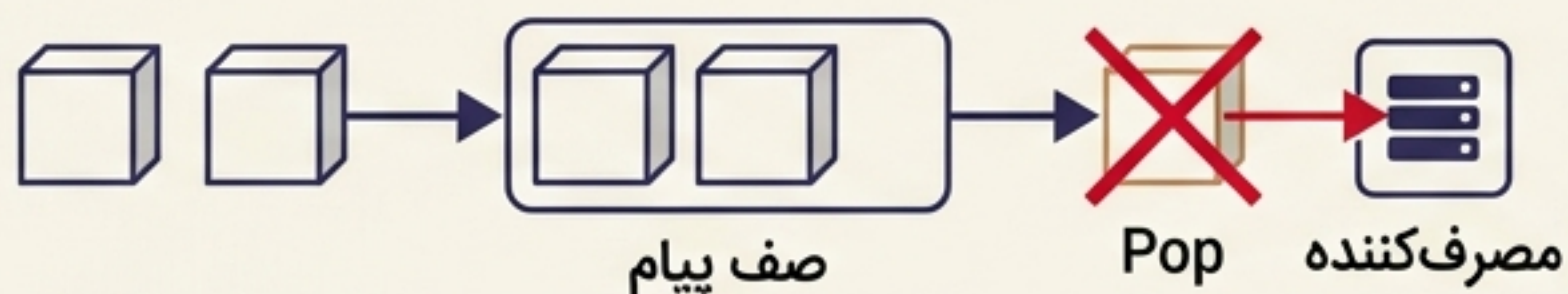
- طراحی شده برای تیم معماران ارشد نرم‌افزار (Senior Architects).

- تمرکز بر تضمین تحویل پیام (Reliability)، مقیاس‌پذیری و الگوهای تراکنشی.

- بررسی چالش‌های یکپارچگی داده‌ها در سیستم‌های توزیع‌شده بانکی.

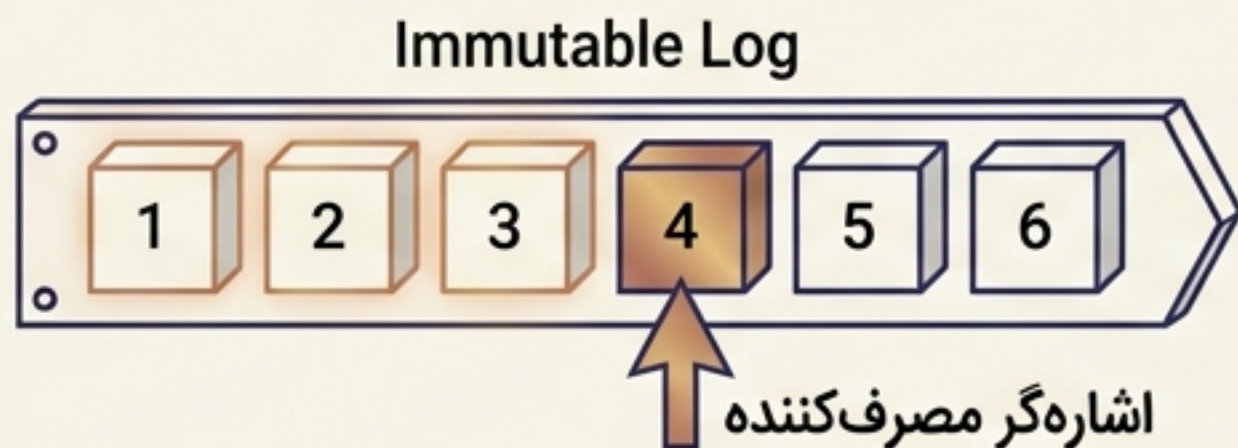
فراتر از صف پیام: کافکا به عنوان Immutable Log

صف پیام سنتی (RabbitMQ)



حافظه موقت - حذف دائمی

لاگ استریمینگ (Kafka)

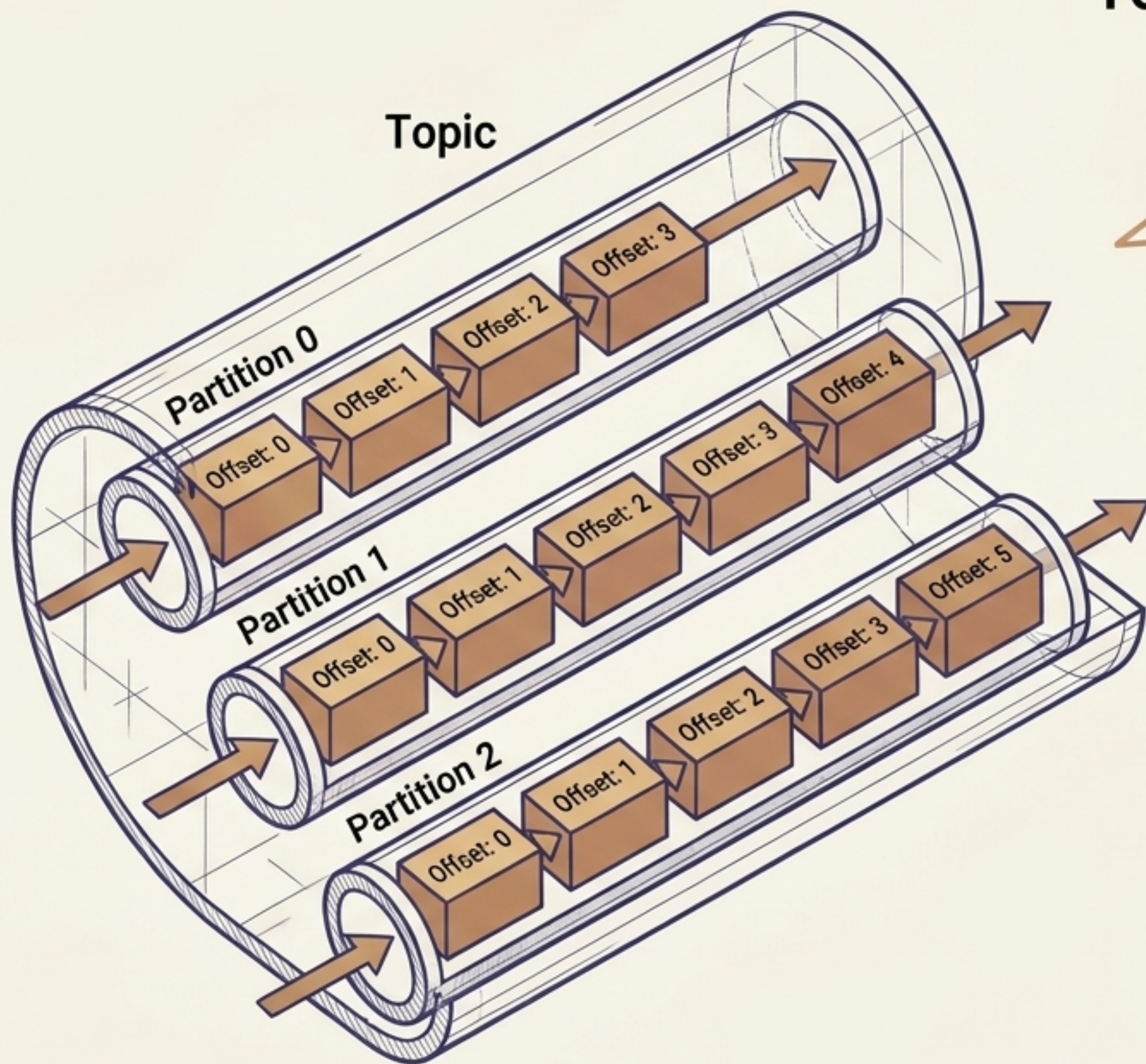


ذخیره سازی دائمی - نگهداری داده

معماری کافکا بر پایه ذخیره سازی دائمی دیسک و دسترسی متوالی (Sequential) بنا شده است، نه حافظه موقت فرار.

- ذخیره سازی غیر قابل تغییر (Immutable Log): رویدادها پس از ثبت، هرگز تغییر نمی کنند (ویژگی حیاتی برای Auditing مالی).
- پرفورمنس در سطح سیستم عامل: بهره گیری از Page Cache و Sequential I/O جهت دور زدن محدودیت های سرعت دیسک.
- استقلال کامل (Decoupling): عدم وابستگی نرخ تولید پیام به سرعت پردازش در سرویس های گیرنده.

آناتومی توزیع داده: Topic, Partition, Offset

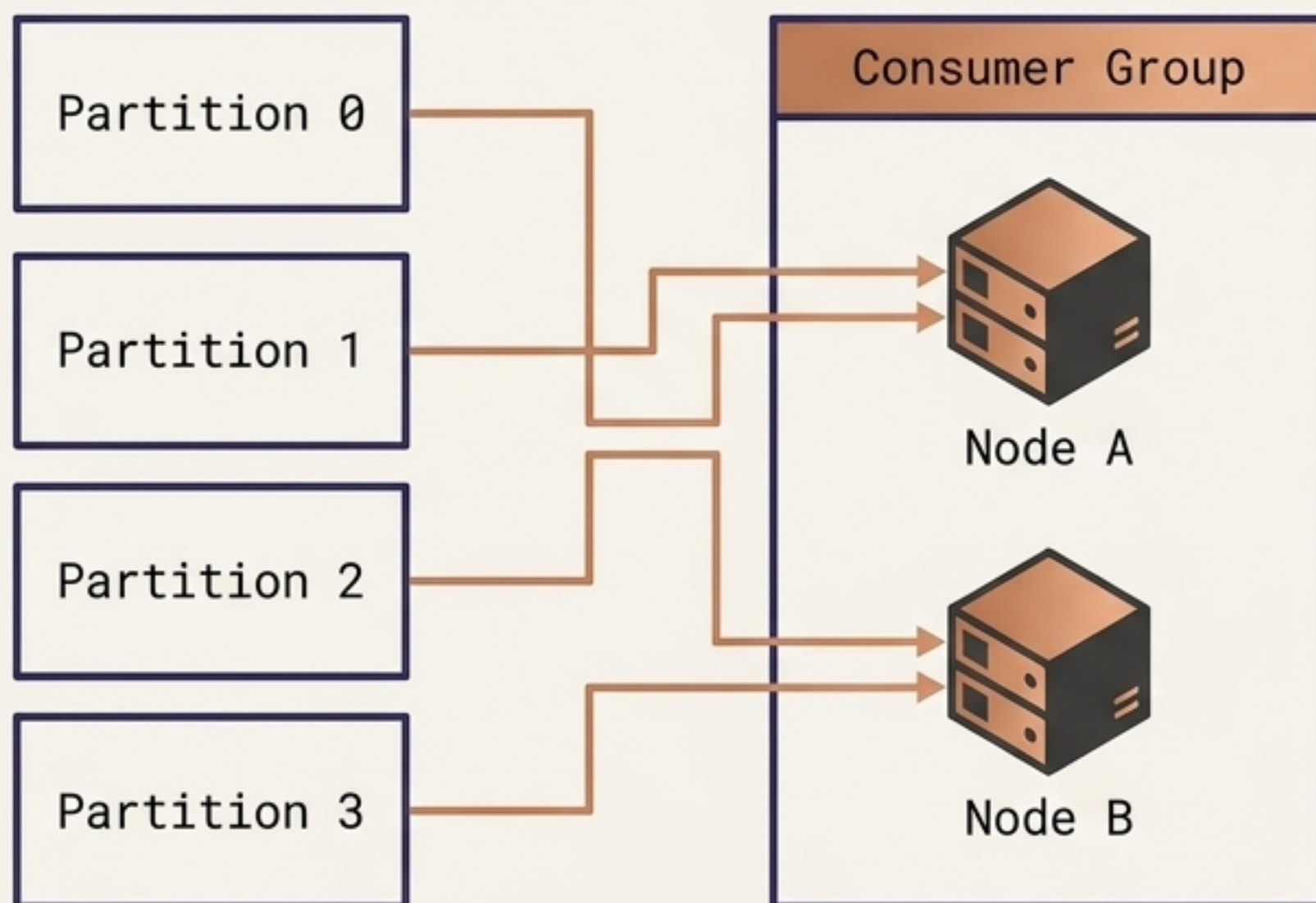


پارتیشن‌ها واحد اصلی مقیاس‌پذیری فیزیکی هستند و افست‌ها وضعیت قطعی هر پیام را ردیابی می‌کنند.

- Topic (تاپیک): دسته‌بندی منطقی جریان‌های داده (مثلاً transfer-requests).
- Partition (پارتیشن): شکستن فیزیکی تاپیک برای پردازش موازی و توزیع بار روی چندین سرور.
- Offset (افست): شناسه عددی و ترتیبی هر پیام در یک پارتیشن؛ قطب‌نمای مصرف‌کننده در اقیانوس داده‌ها.

تضمین پردازش انحصاری با Consumer Groups

گروه‌های مصرف‌کننده، کلیدی‌ترین مکانیزم برای جلوگیری از پردازش تکراری تراکنش‌ها در کلاسترهای توزیع‌شده هستند.



- **تخصیص انحصاری:** هر پارتیشن در یک زمان مشخص، تنها توسط یک عضو از Consumer Group خوانده می‌شود.
- **جلوگیری از رقابت (Race Condition):** رفع قطعی خطر پردازش همزمان یک درخواست انتقال وجه توسط دو کانتینر مختلف.
- **توزیع بار خودکار:** با اضافه شدن نودهای جدید، کافکا پارتیشن‌ها را به صورت خودکار بازتوزیع (Rebalance) می‌کند.

پایداری داده‌ها: Idempotency و دقت محاسباتی

در معماری بانکی، از دست رفتن داده یا ثبت تراکنش تکراری (Double-Spending) غیرقابل قبول است و نیازمند تنظیمات صریح است.

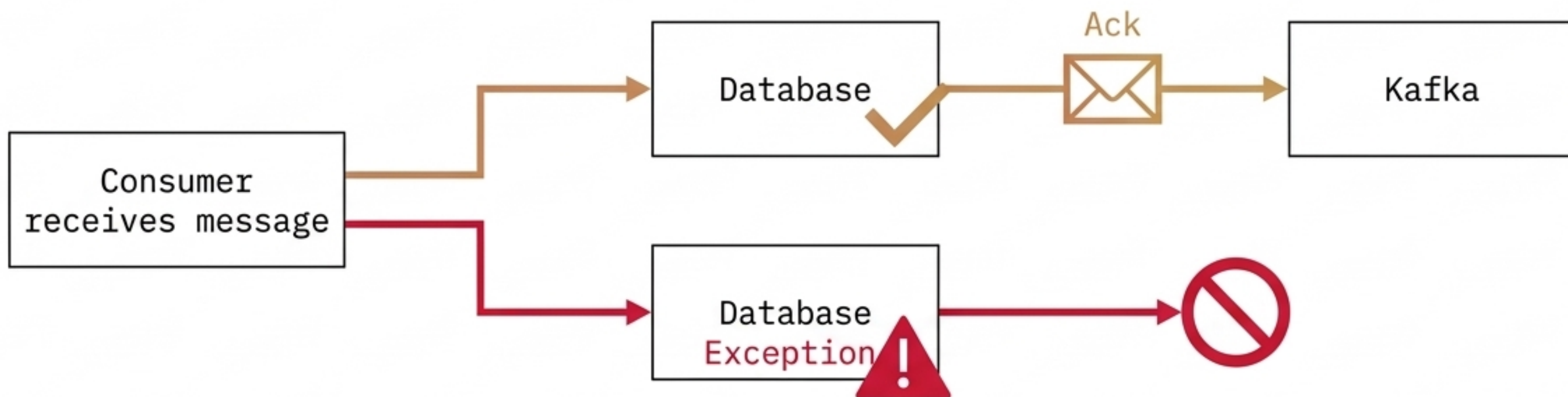


- تولیدکننده ایمن (Idempotent Producer): تخصیص شناسه یکتا با `enable.idempotence=true` جهت انواگ نرساری جهت جلوگیری از ثبت تکراری پیام در قطعی‌های شبکه.
- دقت پولی در حافظه: استفاده الزامی از آبجکت‌های `BigDecimal` برای تمامی مبالغ در سطح کد.
- انتقال امن (Serialization): تبدیل مبالغ مالی به رشته متنی (String) در پکت‌های JSON جهت خنثی‌سازی خطای اعشاری (Rounding Error).

قانون طلایی معماری بانکی: Manual Commit

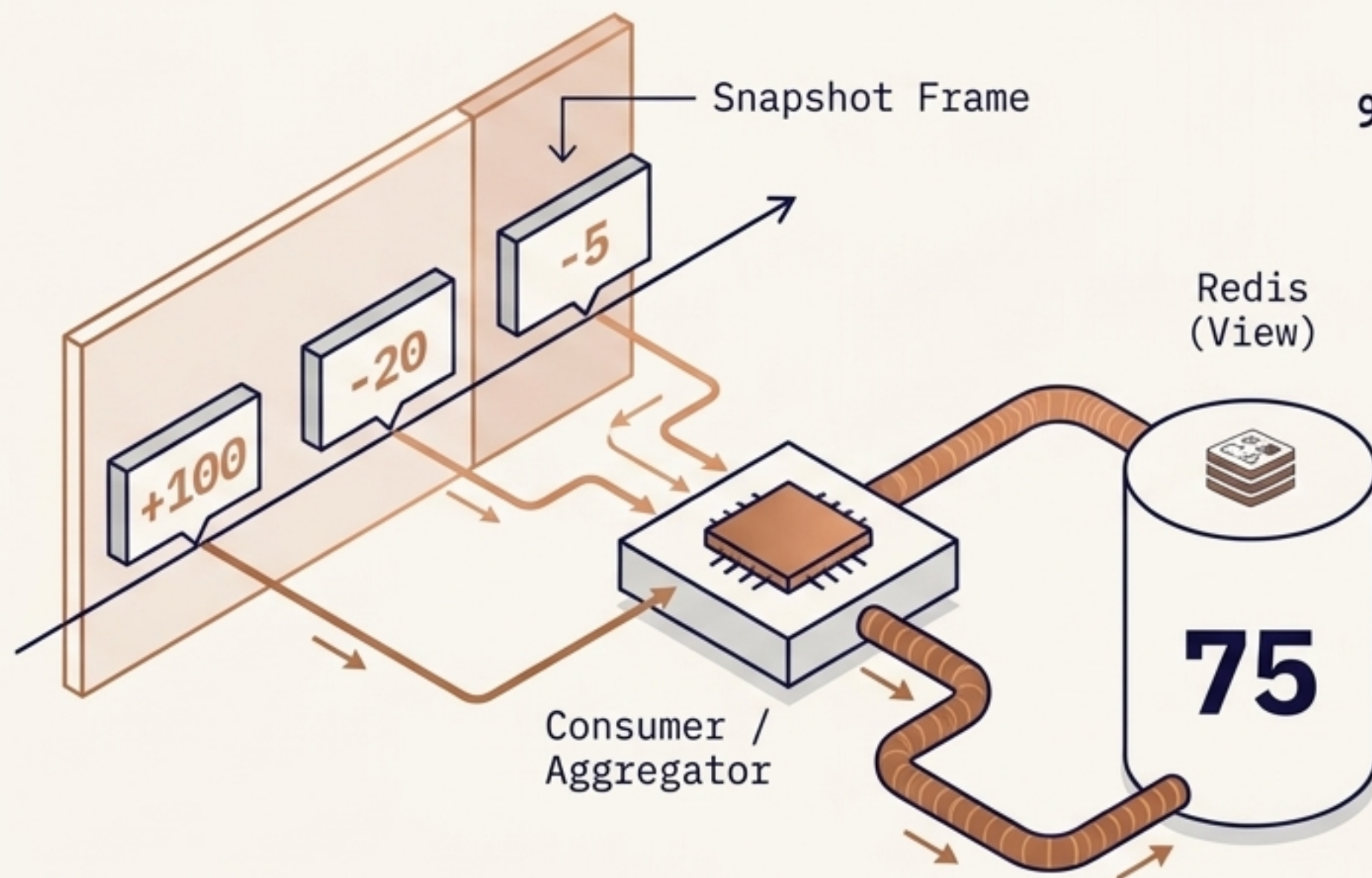
افست پیام تنها زمانی باید تأیید شود که وضعیت نهایی تراکنش در دیتابیس بک‌اند کاملاً تثبیت شده باشد.

- غیرفعال‌سازی Auto Commit: سپردن کنترل تأیید پیام به لایه منطق کسب‌وکار (Business Logic).
- اجرای اتمیک (Atomic Execution): ابتدا ذخیره تراکنش در دیتابیس بانکی، سپس فراخوانی متد Acknowledgment.
- استراتژی جبران (Fault Tolerance): در صورت بروز Exception در دیتابیس، افست کامیت نمی‌شود تا پیام برای پردازش مجدد (Retry) در کافکا باقی بماند.



Event Sourcing و CQRS در معماری حساب‌ها

وضعیت موجودی یک حساب، یک عدد استاتیک نیست؛ بلکه حاصل جمع منطقی تمامی رویدادهای تاریخی آن حساب است.

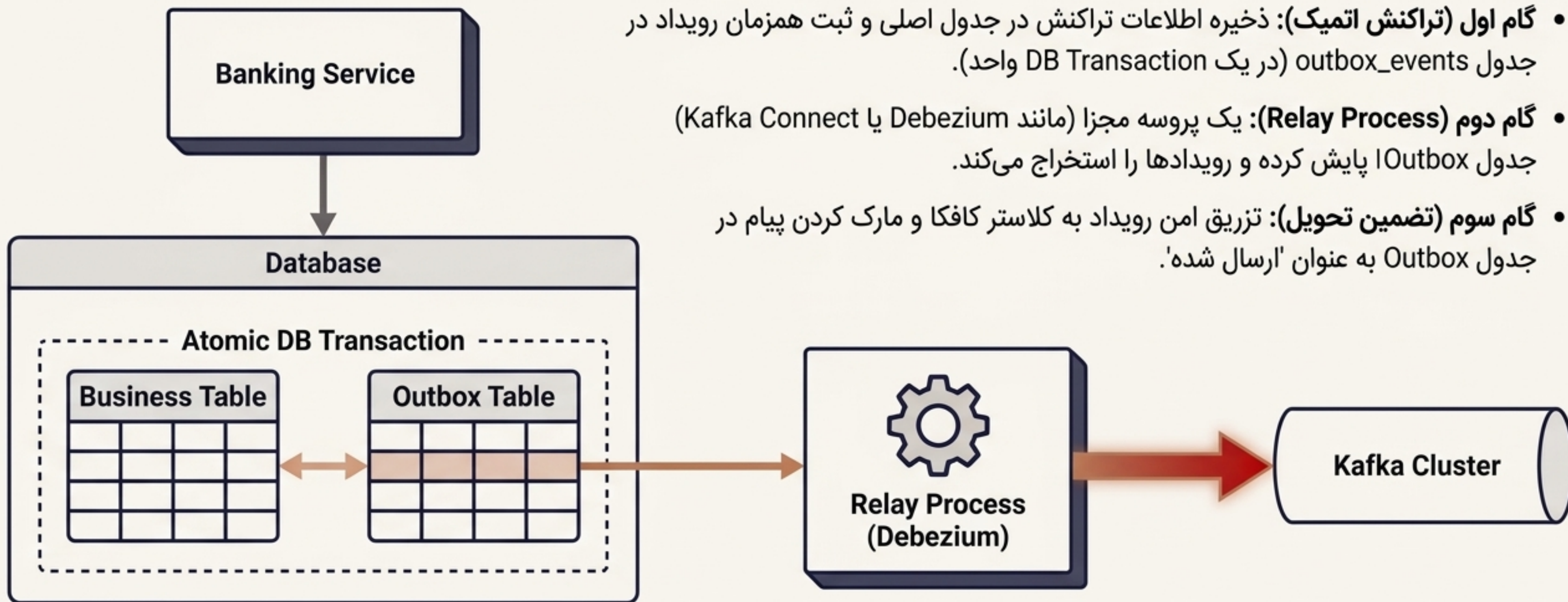


- **لاگ رویدادها (Command):** تمامی واریزها، برداشتها و کارمزدها به عنوان رویدادهای غیرقابل‌تغییر در کافکا ثبت می‌شوند.
- **تفکیک خواندن/نوشتن (CQRS):** سرویس‌های کوئری، رویدادها را مصرف کرده و موجودی را در Redis یا Materialized Views به‌روز می‌کنند.
- **بهینه‌سازی با Snapshot:** تهیه تصویر لحظه‌ای از وضعیت حساب‌ها در بازه‌های زمانی برای جلوگیری از بازخوانی صدها رویداد تاریخی.

غلبه بر Dual-Write: الگوی Outbox Transactional

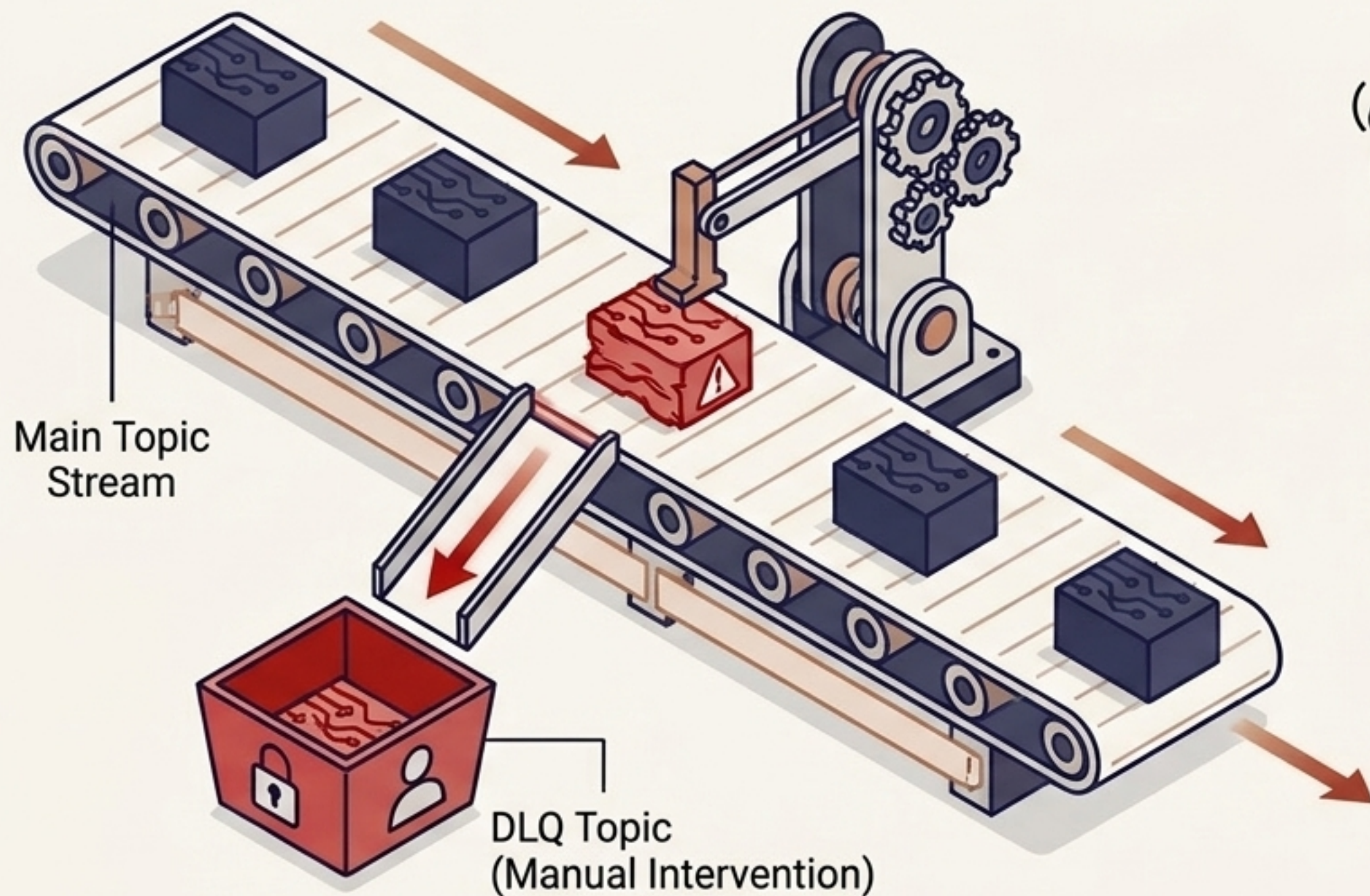
آپدیت همزمان دیتابیس رابطه‌ای و ارسال پیام به کافکا، بدون استفاده از الگوی Outbox، ریسک قطعی ناهماهنگی داده‌ها را به همراه دارد.

- **گام اول (تراکنش اتمیک):** ذخیره اطلاعات تراکنش در جدول اصلی و ثبت همزمان رویداد در جدول outbox_events (در یک DB Transaction واحد).
- **گام دوم (Relay Process):** یک پروسه مجزا (مانند Debezium یا Kafka Connect) جدول Outbox را پایش کرده و رویدادها را استخراج می‌کند.
- **گام سوم (تضمین تحویل):** تزریق امن رویداد به کلاستر کافکا و مارک کردن پیام در جدول Outbox به عنوان 'ارسال شده'.



تابآوری سیستم: مدیریت Poison Pill و DLQ

پیام‌های نامعتبر ساختاری نباید شریان اصلی پردازش سیستم بانکی را مسدود کرده و باعث از کار افتادن کانسیومرها شوند.



- **بحران Poison Pill:** پیام‌هایی (مانند JSON ناقص) که باعث Exception دائمی شده و فرآیند Polling را قفل می‌کنند.
- **سیاست Retry Backoff:** تعیین حداکثر دفعات تلاش مجدد با تأخیر تصاعدی، پیش از تسلیم شدن.
- **مسیریابی به Dead Letter Queue:** انتقال پیام‌های سمی کمی غیرقابل پردازش به انتقال مجزا جدا جدا. غیرقابل پردازش به یک تاپیک مجزا (DLQ) جهت بررسی دستی و آزاد شدن مسیر اصلی.

تجلی معماری در کد (Node.js & Spring Boot)

پیگر بندی صحیح کلاینتها، مرز بین یک سیستم بانکی کاملاً پایدار و یک سیستم شکننده است.

Spring Boot (Java) Consumer

```
@KafkaListener(topics = "transfer-requests",
    groupId = "transaction-processor")
public void processTransaction(String
    message, Acknowledgment ack) {

    try {
        saveToDatabase(message);
        ack.acknowledge();
    } catch (Exception e) {
        System.err.println("Transaction
            failed, offset NOT committed!");
    }
}
```

تأیید
انحصاری
پس از
ذخیره موفق
دیتابیس

Node.js (kafkajs) Consumer

```
const { Kafka } = require('kafkajs');

const kafka = new Kafka({
    clientId: 'esmatiz-fraud-service',
    brokers: ['localhost:9092'] });

const consumer = kafka.consumer({
    groupId: 'fraud-detector' });

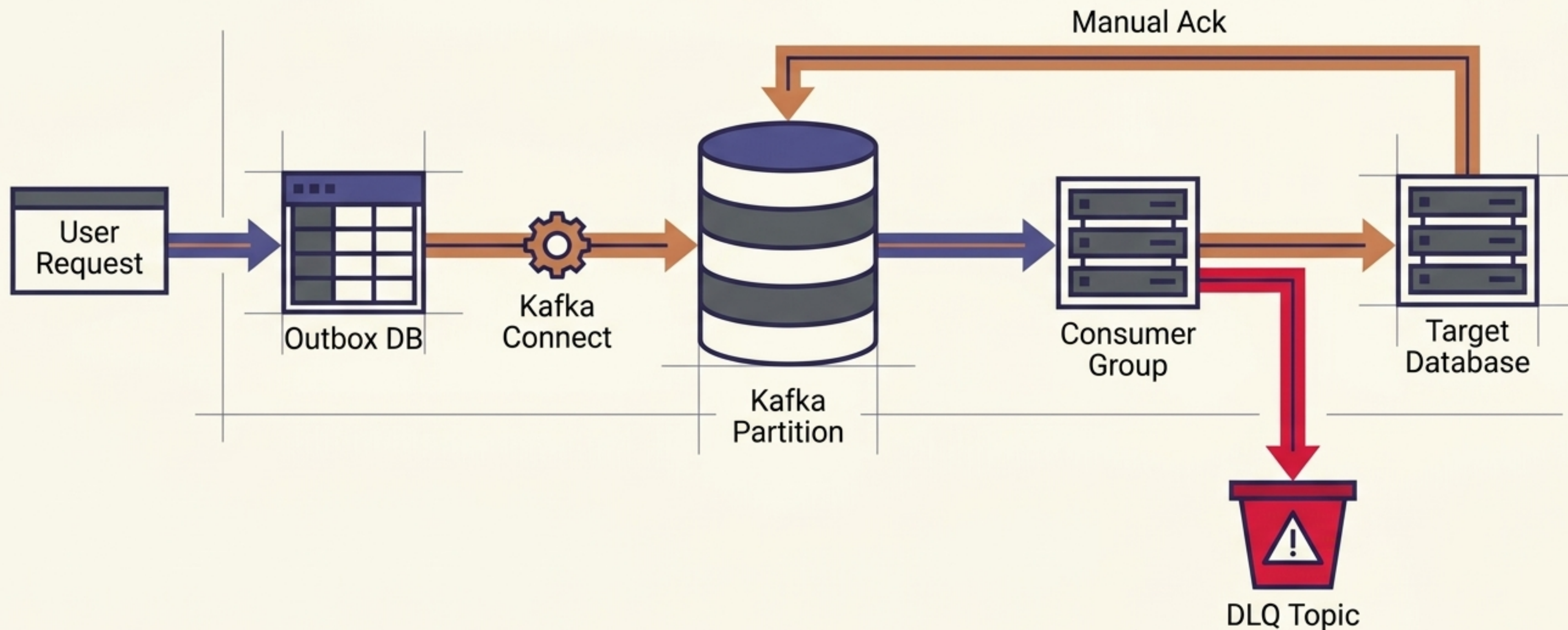
await consumer.connect();
await consumer.subscribe({
    topic: 'transfer-requests' });

await consumer.run({
    eachMessage: async ({ message }) => {
        /* check fraud */
    });
```

عضویت در
گروه
برای کشف
تقلب موازی

تصویر جامع: جریان یکپارچه یک تراکنش بانکی

الگوهای معرفی شده در انزوا کار نمی‌کنند؛ ترکیب آن‌ها یک اکوسیستم مقاوم در برابر خطا می‌سازد.



چک‌لیست معماران سیستم (Key Tenets)

طراحی سیستم‌های مالی بر بستر کافکا نیازمند رعایت دقیق قوانین یکپارچگی است؛ هیچ میانبری وجود ندارد.

✓ هرگز از `Auto Commit` استفاده نکنید؛ کامیت منحصراً پس از تثبیت دیتابیس.

✓ تولیدکننده را ایمن کنید (`enable.idempotence=true`) تا از دابل‌اسپندینگ جلوگیری شود.

✓ مبالغ مالی را همیشه به صورت `String` در پکت ارسال کنید (خنثی‌سازی `Rounding Error`).

✓ استراتژی `DLQ` را برای جلوگیری از قفل شدن فرآیند `Polling` پیاده‌سازی کنید.